C# / .NET Interview Handbook

Candidate + Interviewer Edition — Gated Funnel Format

> **How to use:** Read top-to-bottom. Start at Gate 1. If the candidate struggles, **stop** — no point going deeper. If they pass, proceed to the next gate. Each gate progressively tests deeper understanding.

TABLE OF CONTENTS

GATE 1 — Fundamentals (Junior: 0-3 years)

- Q1. What is .NET? How is it different from C#?
- Q2. Explain the evolution: .NET Framework → .NET Core → .NET 5/6/7/8.
- Q3. What is the difference between value types and reference types?
- Q4. What are the four pillars of Object-Oriented Programming?
- Q5. Why are strings immutable in C#?
- Q6. When should you use `StringBuilder` vs string concatenation?
- Q7. What is the difference between `Array` and `List<T>`?
- Q8. What are nullable types in C#?
- Q9. What is the difference between `var`, `object`, and `dynamic`?
- Q10. Explain exception handling in C#.
- Q11. What are access modifiers in C#?
- Q12. What is the difference between `==` and `.Equals()`?
- Q13. What is `IEnumerable<T>` and why is it important?
- Q14. What is the difference between `const` and `readonly`?
- Q15. What is the `using` statement for?

GATE 2 — Working Knowledge (Mid: 3-7 years)

- Q16. How does C# code go from source to execution?
- Q17. What is the CLR and what does it do?
- Q18. What is boxing and unboxing? Why should you care?
- Q19. Explain delegates, events, and lambdas.
- Q20. What is LINQ and how does deferred execution work?
- Q21. What is `yield return` and how does it work?
- Q22. Constructors — chaining, static, and private.
- Q23. Abstract class vs Interface — when do you choose which?
- Q24. Explain generics. Why do they exist?
- Q25. What does `async/await` actually do?
- Q26. Scenario: Your colleague wrote this code. What's wrong?

GATE 3 — Deep Understanding (Senior: 7-15 years)

- Q27. Explain how the Garbage Collector works internally.

- Q28. Scenario: Your production API has memory that keeps growing but never crashes. What's happening?
- Q29. Implement the correct IDisposable pattern.
- Q30. What happens at the compiler level when you write `async/await`?
- Q31. Scenario: Your web API handles 500 req/sec fine, but at 2000 req/sec, response times spike to 30 seconds. CPU is at 20%. What's happening?
- Q32. Explain thread safety in C#. What are the options?
- Q33. Structs vs classes — when should you use structs? What are the dangers?
- Q34. What are `Span<T>` and `Memory<T>`? When would you use them?
- Q35. What are records? Pattern matching? What's new in modern C# (9-12)?
- Q36. Scenario: You're reviewing code and find this. What are the issues?
- Q37. What is reflection and when would you use it (and when would you avoid it)?

GATE 4 — Architecture & Leadership (Lead/Architect: 15+ years)

- Q38. Scenario: You're the tech lead for a .NET Framework 4.8 application with 200+ developers. The CTO wants to migrate to .NET 8. How do you approach this?
- Q39. How does JIT tiered compilation work in modern .NET? Why does it matter?
- Q40. Scenario: You're designing the performance profiling strategy for a production .NET API that handles financial transactions. What's your approach?
- Q41. GC tuning — when and how would you tune the garbage collector?
- Q42. How would you design a plugin architecture in .NET?
- Q43. Scenario: "We should rewrite the system in [language X]." How do you evaluate this as an architect?

APPENDIX — Quick Reference Card

GATE 1 — Fundamentals `[Junior: 0-3 years]`

> **Question style:** Mostly direct conceptual questions. Testing basic understanding of the language and platform. **Pass criteria:** Candidate can explain core concepts clearly with simple examples.

Q1. What is .NET? How is it different from C#?

Expected Answer:

- **.NET** is a **platform/framework** — a runtime + libraries for building applications.
- **C#** is a **programming language** that runs on the .NET platform.
- Analogy: C# is like English (the language), .NET is like the country's infrastructure (roads, laws, services) you operate in.

What to listen for: Candidate distinguishes between language and platform. **Red flag:** Treats C# and .NET as the same thing.

Q2. Explain the evolution: .NET Framework → .NET Core → .NET 5/6/7/8.

Expected Answer:

Era	What	Key Facts
.NET Framework (2002-2019)	Original, Windows-only	Tied to Windows, IIS, System.Web. Versions 1.0 to 4.8
.NET Core (2016-2020)	Cross-platform rewrite	Runs on Windows/Linux/macOS. Lightweight, modular, open-source
.NET 5+ (2020-present)	Unified platform	Merged Framework + Core. One .NET going forward. Yearly releases

```
.NET Framework 4.8 (last version — maintenance only)
        ↓ (rewrite)
.NET Core 1.0 → 2.0 → 3.1 (LTS)
        ↓ (rebrand + unify)
.NET 5 → 6 (LTS) → 7 → 8 (LTS) → 9
```

Scenario (for probing): "*Your company has a .NET Framework 4.8 app. Management wants to 'upgrade to .NET 8'. What do you tell them?*"

What to listen for: It's not a simple upgrade — it's a **migration**. Many APIs changed, `System.Web` doesn't exist, need to evaluate dependencies, plan incrementally. **Red flag:** "Just change the target framework in the project file."

Q3. What is the difference between value types and reference types?

Expected Answer:

Aspect	Value Type	Reference Type
Contains	The actual data	A reference (pointer) to data on the heap
Assignment	Copies the data	Copies the reference (same object)
Default value	Zeros (`0`, `false`, etc.)	`null`
Examples	`int`, `bool`, `double`, `struct`, `enum`	`string`, `class`, `array`, `object`, `delegate`

```
// Value type — independent copies
int a = 5;
int b = a;     // b is a copy
b = 10;        // a is still 5

// Reference type — shared object
var list1 = new List<int> { 1, 2, 3 };
var list2 = list1;      // Both point to same list
list2.Add(4);           // list1 also has 4!
```

What to listen for: Clear understanding that value types copy data, reference types copy the reference. **Red flag:** "Value types are on the stack, reference types on the heap." (Oversimplified — value types inside a class live on the heap.)

Q4. What are the four pillars of Object-Oriented Programming?

Expected Answer:

1. **Encapsulation** — Hiding internal state behind a public interface. Use properties and methods to control access.
2. **Abstraction** — Showing only what's necessary. A `Car` class exposes `Drive()` — you don't need to know how the engine works.
3. **Inheritance** — A class derives from another, inheriting its behavior. `Dog : Animal`.
4. **Polymorphism** — Same method name, different behavior based on the object. `animal.Speak()` → Dog says "Woof", Cat says "Meow".

```
// Encapsulation
public class BankAccount
{
    private decimal _balance;  // Hidden
    public decimal Balance => _balance;  // Controlled access
    public void Deposit(decimal amount) { if (amount > 0) _balance += amount; }
}

// Polymorphism
Animal animal = new Dog();
animal.Speak();  // "Woof" — runtime polymorphism
```

What to listen for: Can explain each with a practical example, not just definitions. **Red flag:** Only recites definitions without understanding when/why to use each.

Q5. Why are strings immutable in C#?

Expected Answer: Once created, a string's content cannot be changed. Any "modification" creates a **new string** object.

```
string s = "Hello";
s += " World";  // Creates NEW string "Hello World", old "Hello" is garbage
```

Why immutable?

- **Thread safety** — Safe to share across threads without locks.
- **String interning** — Identical string literals share the same memory.
- **Security** — Connection strings, file paths can't be tampered with after validation.

- **Hash code caching** — Used as dictionary keys safely.

What to listen for: Mentions at least 2 reasons. Knows that `+=` in a loop is expensive. **Red flag:** Doesn't know that `+=` creates a new string.

Q6. When should you use `StringBuilder` vs string concatenation?

Expected Answer:

Scenario	Use	Why
2-3 concatenations	`string` or `$""`	Compiler optimizes, negligible overhead
Loop with many concatenations	`StringBuilder`	Avoids N string allocations
Building large text	`StringBuilder`	Mutable buffer, O(1) amortized append

```
// BAD — creates 10,000 intermediate strings
string result = "";
for (int i = 0; i < 10000; i++)
    result += i.ToString();

// GOOD — one mutable buffer
var sb = new StringBuilder();
for (int i = 0; i < 10000; i++)
    sb.Append(i);
string result = sb.ToString();
```

Q7. What is the difference between `Array` and `List<T>`?

Expected Answer:

Feature	Array	List<T>
Size	Fixed at creation	Dynamic (resizes automatically)
Performance	Slightly faster (direct memory access)	Slight overhead (internal array + bounds checking)
API	Basic (indexer, Length)	Rich (Add, Remove, Find, Sort, LINQ)
Memory	Exact size	May have unused capacity

```
int[] arr = new int[5];        // Fixed size — can't add 6th element
List<int> list = new();         // Dynamic — grows as needed
list.Add(1);
list.Add(2);
```

When to use Array: Known fixed size, performance-critical inner loops, interop. **When to use List<T>:** Almost everywhere else — flexibility wins.

Q8. What are nullable types in C#?

Expected Answer:

Nullable value types (`T?`): Allow value types to represent `null`.

```
int? age = null;        // Nullable<int>
int normalAge = 25;     // Cannot be null

if (age.HasValue)
    Console.WriteLine(age.Value);

int result = age ?? 0;  // Null-coalescing: 0 if null
```

Nullable reference types (C# 8+): Compile-time warnings for potential null references.

```
#nullable enable
string name = null;     // Warning!
string? name2 = null;   // OK — explicitly nullable
```

Q9. What is the difference between `var`, `object`, and `dynamic`?

Expected Answer:

Feature	`var`	`object`	`dynamic`
Type decided at	Compile-time (inferred)	Compile-time (explicit)	Runtime
Type safety	Full	Partial (needs casting)	None until runtime
IntelliSense	Yes	Only `object` members	No

```
var x = 42;          // Compiler knows it's int — full type safety
object y = 42;       // Must cast: (int)y — boxing happens
dynamic z = 42;      // No compile-time checking — errors at runtime
z.Foo();             // Compiles! Crashes at runtime.
```

Key insight: `var` is NOT dynamic typing — it's **implicit static typing**.

Q10. Explain exception handling in C#.

Expected Answer:

```
try
{
    var result = Divide(10, 0);
}
catch (DivideByZeroException ex)  // Specific first
{
    _logger.LogWarning("Division by zero: {Message}", ex.Message);
}
catch (Exception ex)  // General last
{
    _logger.LogError(ex, "Unexpected error");
    throw;  // Re-throw preserving stack trace
}
finally
{
    // Always runs — cleanup code
    connection?.Dispose();
}
```

Key rules:

- Catch **specific** exceptions before general.
- Use `throw;` (not `throw ex;`) to preserve the stack trace.
- `finally` always executes — for cleanup.
- Don't catch exceptions you can't handle — let them bubble up.

Q11. What are access modifiers in C#?

Expected Answer:

Modifier	Accessible From
`public`	Everywhere
`private`	Same class only
`protected`	Same class + derived classes
`internal`	Same assembly
`protected internal`	Same assembly OR derived classes (union)
`private protected`	Same assembly AND derived classes (intersection)

Default: Class members are `private`. Classes are `internal`.

Q12. What is the difference between `==` and `.Equals()`?

Expected Answer:

```
// For reference types (default behavior):
var a = new Person("Alice");
var b = new Person("Alice");
a == b;       // False — different objects (reference comparison)
a.Equals(b);  // False — unless Equals() is overridden

// For strings (special case — == is overloaded):
string s1 = "Hello";
string s2 = "Hello";
s1 == s2;     // True — string overloads == for value comparison
s1.Equals(s2); // True

// For value types:
int x = 5, y = 5;
x == y;       // True — value comparison
x.Equals(y);  // True
```

Key rule: `==` is reference equality by default for classes (unless overloaded). `Equals()` can be overridden for custom comparison logic.

Q13. What is `IEnumerable<T>` and why is it important?

Expected Answer: `IEnumerable<T>` is the **base interface for iteration** in .NET. Any type implementing it can be used with `foreach` and LINQ.

```
public interface IEnumerable<T>
{
    IEnumerator<T> GetEnumerator();
}

// Every collection implements it: List<T>, Array, Dictionary<K,V>, HashSet<T>
IEnumerable<int> numbers = new List<int> { 1, 2, 3 };
foreach (var n in numbers) { ... }  // Works because of IEnumerable
numbers.Where(n => n > 1);          // LINQ works because of IEnumerable
```

Why important: It's the common contract for all sequences. Methods accepting `IEnumerable<T>` work with any collection.

Q14. What is the difference between `const` and `readonly`?

Expected Answer:

Feature	`const`	`readonly`
Set when	Compile-time	Runtime (constructor)

Feature	`const`	`readonly`
Inlined	Yes (value baked into calling code)	No (read from field)
Types allowed	Primitives, string, null	Any type
Can change between builds	No (callers need recompile)	Yes (value resolved at runtime)

```
public class Config
{
    public const int MaxRetries = 3;        // Baked into IL at compile time
    public readonly string ApiUrl;           // Set in constructor

    public Config(string url) => ApiUrl = url;
}
```

Gotcha: If you change a `const` value in a library, all consuming assemblies must be recompiled — the old value is inlined in their IL.

Q15. What is the `using` statement for?

Expected Answer: The `using` statement ensures `Dispose()` is called when the scope ends — even if an exception occurs.

```
// Classic using block
using (var connection = new SqlConnection(connStr))
{
    connection.Open();
    // ... use connection
} // connection.Dispose() called automatically here

// Modern using declaration (C# 8+)
using var connection = new SqlConnection(connStr);
connection.Open();
// ... use connection
// Dispose called at end of enclosing scope
```

Why it matters: Resources like database connections, file handles, and HTTP clients must be explicitly released. GC handles memory, but not unmanaged resources.

↑ Back to Index

GATE 2 — Working Knowledge `[Mid: 3-7 years]`

> **Question style:** Mix of direct + scenario-based. Testing how things work internally and practical application. **Pass criteria:** Candidate explains mechanisms, not just definitions. Can troubleshoot

> scenarios.

Q16. How does C# code go from source to execution?

Expected Answer:

```
C# Source (.cs)
    ↓  [Roslyn compiler]
IL Code + Metadata (.dll/.exe)  ← "Assembly"
    ↓  [CLR loads assembly]
IL in memory
    ↓  [JIT compiler — per method, on first call]
Native machine code
    ↓  [CPU]
Running application
```

Two-phase compilation:

1. **Build time:** Source → IL (catches syntax/type errors).
2. **Runtime:** IL → native code via JIT (optimized for actual CPU).

Why two phases? Platform independence (same IL on Windows/Linux), runtime optimization (JIT knows the actual CPU), language interop (C#, F#, VB all produce IL).

What to listen for: Understands two-phase compilation, knows what JIT means. **Red flag:** Thinks C# compiles directly to machine code.

Q17. What is the CLR and what does it do?

Expected Answer: CLR (Common Language Runtime) is the **execution engine** for .NET. It provides:

1. **JIT Compilation** — IL → native code
2. **Garbage Collection** — Automatic memory management
3. **Type Safety** — Verifies IL, prevents invalid memory access
4. **Exception Handling** — Structured exception handling across languages
5. **Thread Management** — ThreadPool, synchronization
6. **Assembly Loading** — Finding and loading dependencies

Analogy: CLR is to .NET what the JVM is to Java — a managed runtime between your code and the OS.

Q18. What is boxing and unboxing? Why should you care?

Expected Answer:

Boxing: Wrapping a value type in an `object` → heap allocation. **Unboxing:** Extracting the value type from the `object` → type check + copy.

```
int num = 42;
object boxed = num;         // Boxing: heap allocation
int unboxed = (int)boxed;   // Unboxing: type check + copy
```

Scenario: "*Your application is doing 1 million iterations per second and you notice high GC pressure. You find this code:*"

```
ArrayList list = new ArrayList();
for (int i = 0; i < 1_000_000; i++)
    list.Add(i);  // Boxing every int!
```

What to listen for: Candidate identifies the boxing and suggests `List<int>` (generic, no boxing). **Red flag:** Doesn't know boxing allocates on the heap.

Q19. Explain delegates, events, and lambdas.

Expected Answer:

Delegate: A type-safe function pointer — holds reference to a method.

```
Func<int, int, int> add = (a, b) => a + b;
int result = add(3, 4);  // 7
```

Event: A restricted delegate — only the owner can invoke it.

```
public class Button
{
    public event EventHandler Clicked;
    public void OnClick() => Clicked?.Invoke(this, EventArgs.Empty);
}

button.Clicked += (s, e) => Console.WriteLine("Clicked!");
// button.Clicked(this, args);  // Won't compile! Can't invoke from outside
```

Lambda: Shorthand for anonymous methods.

```
Func<int, bool> isEven = x => x % 2 == 0;      // Expression lambda
Action<string> log = msg => { Console.WriteLine(msg); };  // Statement lambda
```

Built-in delegates:

- `Func<T, TResult>` — has return value

- `Action<T>` — no return value (void)
- `Predicate<T>` — returns bool

Q20. What is LINQ and how does deferred execution work?

Expected Answer:

LINQ (Language Integrated Query) provides query syntax for collections.

```
var numbers = new List<int> { 1, 2, 3, 4, 5 };
var query = numbers.Where(n => n > 2).Select(n => n * 10);
// NO execution yet! Just a query definition.

numbers.Add(6);  // Modify source AFTER defining query

foreach (var n in query)
    Console.Write($"{n} ");  // 30 40 50 60 — includes 6!
```

Deferred: `Where`, `Select`, `OrderBy`, `Take`, `Skip` — execute when iterated. **Immediate:** `ToList()`, `ToArray()`, `Count()`, `First()`, `Any()` — execute immediately.

Scenario: *"A developer writes this and complains it's slow. What's the issue?"*

```
var users = GetUsersFromDatabase().Where(u => u.IsActive);
var count = users.Count();    // Enumerates the query
var list = users.ToList();     // Enumerates AGAIN!
```

What to listen for: Identifies double enumeration. Suggests materializing once with `ToList()`.

Q21. What is `yield return` and how does it work?

Expected Answer:

`yield return` creates an **iterator** — the compiler generates a state machine that produces values lazily.

```
public IEnumerable<int> EvenNumbers(int max)
{
    for (int i = 0; i <= max; i++)
    {
        if (i % 2 == 0)
            yield return i;  // Pause here, return value
        // Execution resumes here on next MoveNext()
    }
}

// Usage — only generates values as consumed
```

```
foreach (var n in EvenNumbers(1000000))
{
    if (n > 10) break;  // Only generated 0,2,4,6,8,10 — not all million
}
```

Key benefit: Memory efficient for large/infinite sequences. Values produced on demand.

Q22. Constructors — chaining, static, and private.

Expected Answer:

```
public class Person
{
    public string Name { get; }
    public int Age { get; }

    // Constructor chaining with 'this'
    public Person() : this("Unknown", 0) { }
    public Person(string name) : this(name, 0) { }
    public Person(string name, int age)  // Primary
    {
        Name = name;
        Age = age;
    }

    // Static constructor — runs once, before first use, thread-safe
    static Person()
    {
        // Initialize static state
    }
}
```

Private constructor uses:

- **Singleton pattern:** Prevent external instantiation
- **Factory methods:** Control how objects are created
- **Static-only classes:** (prefer `static class` in modern C#)

Gotcha: If a static constructor throws, the type is **permanently unusable** (`TypeInitializationException`).

Q23. Abstract class vs Interface — when do you choose which?

Expected Answer:

Need	Abstract Class	Interface
Shared implementation	Yes	Limited (C# 8+ default methods)

Need	Abstract Class	Interface
Fields/state	Yes	No
Constructor	Yes	No
Multiple inheritance	No (one base class)	Yes (many interfaces)
Access modifiers	All	Public only (pre-C# 8)

Choose abstract class when: Shared state/behavior + "is-a" relationship. **Choose interface when:** Multiple unrelated types share a contract + "can-do" capability.

Scenario: "*You're designing a tax calculation system. You have FederalTax, StateTax, and LocalTax. They all share a base algorithm but calculate deductions differently. Abstract class or interface?*"

What to listen for: Abstract class — shared algorithm (Template Method pattern), with `abstract` method for deductions. The shared code prevents duplication.

Q24. Explain generics. Why do they exist?

Expected Answer:

Generics provide **type safety + code reuse** without boxing or casting.

```
// Without generics (pre-C# 2.0) — boxing, no type safety
ArrayList list = new ArrayList();
list.Add(42);           // Boxing int → object
list.Add("hello");      // No error! Type mismatch at runtime
int num = (int)list[0]; // Must cast, could throw

// With generics — type safe, no boxing
List<int> list = new List<int>();
list.Add(42);           // No boxing
// list.Add("hello");   // Compile error! Type safety
int num = list[0];      // No cast needed
```

Generic class/method:

```
public class Repository<T> where T : class, IEntity
{
    public T GetById(int id) { ... }
    public void Save(T entity) { ... }
}

// Constraints limit what T can be:
// where T : class          — reference type only
// where T : struct         — value type only
// where T : new()          — must have parameterless constructor
```

```
// where T : IEntity        — must implement interface
// where T : BaseClass      — must derive from class
```

Q25. What does `async/await` actually do?

Expected Answer:

`async/await` enables **non-blocking I/O** — the thread is freed while waiting for I/O to complete.

```
public async Task<string> GetDataAsync()
{
    // Thread is FREE while waiting for HTTP response
    var response = await httpClient.GetAsync("https://api.example.com");

    // Continues here when response arrives (possibly on a different thread)
    var content = await response.Content.ReadAsStringAsync();
    return content;
}
```

Key points:

- `await` does NOT create a new thread.
- It registers a continuation and **releases the current thread**.
- When the I/O completes, execution resumes (possibly on a different thread).
- If the awaited task is already complete, no suspension occurs.

Scenario: "*A junior developer says 'async makes my code run in parallel.' Is that correct?*"

What to listen for: No — async is about non-blocking, not parallelism. For CPU-bound parallel work, use `Task.Run()` or `Parallel.ForEach()`.

Q26. Scenario: Your colleague wrote this code. What's wrong?

```
public class OrderService
{
    public void ProcessOrder(Order order)
    {
        var result = GetOrderDetailsAsync(order.Id).Result;
        // ... process result
    }
}
```

Expected Answer: `.Result` **blocks the calling thread** — this is the **sync-over-async anti-pattern**.

Problems:

1. Blocks a ThreadPool thread, reducing throughput.
2. Can cause **deadlock** in ASP.NET Framework (SynchronizationContext).
3. In ASP.NET Core, no deadlock but still wastes a thread (ThreadPool starvation risk).

Fix:

```
public async Task ProcessOrderAsync(Order order)
{
    var result = await GetOrderDetailsAsync(order.Id);
    // ... process result
}
```

Rule: Async all the way — from controller to data layer.

↑ Back to Index

GATE 3 — Deep Understanding [Senior: 7-15 years]

> **Question style:** Mostly scenario-based. Present real problems, expect diagnosis and trade-off analysis.
> **Pass criteria:** Candidate reasons about internals, trade-offs, and production implications.

Q27. Explain how the Garbage Collector works internally.

Expected Answer:

```
Managed Heap
├── Small Object Heap (SOH)
│   ├── Gen 0 – New objects (~256 KB, collected frequently)
│   ├── Gen 1 – Survived 1 GC (buffer zone, ~2 MB)
│   └── Gen 2 – Long-lived objects (collected rarely, expensive)
├── Large Object Heap (LOH)
│   └── Objects ≥ 85,000 bytes (collected with Gen2, NOT compacted by default)
└── Pinned Object Heap (POH) – .NET 5+
```

Generational hypothesis: Most objects die young.

How GC works:

1. **Mark** — Starting from GC roots (stack, statics, GC handles), trace all reachable objects.
2. **Sweep** — Unreachable objects are garbage.
3. **Compact** — Move survivors together, update references (SOH only, LOH not compacted by default).

Gen0 collection: Fast (~1ms), frequent. Survivors promoted to Gen1. **Gen2 collection:** Slow (10-100ms), rare. Full GC — includes LOH.

GC triggers: Gen0 threshold exceeded, system memory pressure, `GC.Collect()` (avoid!).

What to listen for: Understands generations, why Gen2 is expensive, what GC roots are.

Q28. Scenario: Your production API has memory that keeps growing but never crashes. What's happening?

Expected Answer: This is a **managed memory leak** — objects are reachable but unwanted.

Common causes:

1. **Event handler leak:** `publisher.Event += handler;` never unsubscribed — publisher holds subscriber alive.
2. **Static collections:** `static List<T> cache` grows forever without eviction.
3. **Closure captures:** Lambda captures `this`, keeping the entire object alive.
4. **Timer leak:** `System.Timers.Timer` not disposed — holds references via events.
5. **Long-lived `ConcurrentDictionary`:** Used as cache without expiry.

Diagnosis approach:

1. Take a memory dump (`dotnet-dump collect`).
2. Analyze with `dotnet-dump analyze` or Visual Studio.
3. Look at Gen2 heap — what's accumulating?
4. Check the GC root chain — why are objects still reachable?

What to listen for: Candidate identifies that GC only collects unreachable objects. Knows diagnostic tools.
Red flag: "Call `GC.Collect()`" or "increase memory."

Q29. Implement the correct IDisposable pattern.

Expected Answer:

```
public class ResourceHolder : IDisposable
{
    private IntPtr _unmanagedHandle;
    private ManagedStream _managedResource;
    private bool _disposed;

    public void Dispose()
    {
        Dispose(disposing: true);
        GC.SuppressFinalize(this);  // No need to finalize — we cleaned up
    }

    protected virtual void Dispose(bool disposing)
    {
        if (_disposed) return;

        if (disposing)
        {
```

```
            // Free MANAGED resources
            _managedResource?.Dispose();
        }

        // Free UNMANAGED resources (always)
        CloseHandle(_unmanagedHandle);
        _disposed = true;
    }

    ~ResourceHolder()  // Finalizer — safety net
    {
        Dispose(disposing: false);  // Only clean unmanaged
    }
}
```

Why `GC.SuppressFinalize`? Finalizable objects survive an extra GC cycle (promoted to Gen1+). Suppressing finalization after Dispose lets the GC collect normally.

Modern simplification (no unmanaged resources):

```
public class SimpleResource : IDisposable
{
    private SqlConnection? _connection;

    public void Dispose()
    {
        _connection?.Dispose();
        _connection = null;
    }
}
```

Q30. What happens at the compiler level when you write `async/await`?

Expected Answer:

The compiler transforms the async method into a **state machine struct**:

```
// What you write:
public async Task<string> GetDataAsync()
{
    var response = await httpClient.GetAsync("url");    // State 0
    var content = await response.Content.ReadAsStringAsync(); // State 1
    return content;
}

// What the compiler generates (simplified):
private struct GetDataAsyncStateMachine : IAsyncStateMachine
{
```

```
    public int _state;  // -1=start, 0=awaiting GetAsync, 1=awaiting ReadAsString
    public AsyncTaskMethodBuilder<string> _builder;
    // All local variables become fields on the struct

    public void MoveNext()
    {
        switch (_state)
        {
            case -1:  // Start
                var awaiter = httpClient.GetAsync("url").GetAwaiter();
                if (!awaiter.IsCompleted)
                {
                    _state = 0;
                    _builder.AwaitUnsafeOnCompleted(ref awaiter, ref this);
                    return;  // ← Returns to caller! Thread is free.
                }
                goto case 0;
            case 0:  // GetAsync completed
                var response = awaiter.GetResult();
                // ... continue to next await
        }
    }
}
```

Key insight: Each `await` is a potential **suspension point**. The state machine remembers where to resume.

Q31. Scenario: Your web API handles 500 req/sec fine, but at 2000 req/sec, response times spike to 30 seconds. CPU is at 20%. What's happening?

Expected Answer:

Low CPU + high latency = **ThreadPool starvation**.

Diagnosis:

- Threads are **blocked** (not doing CPU work, hence low CPU).
- Likely cause: sync-over-async calls (`.Result`, `.Wait()`) blocking ThreadPool threads.
- ThreadPool grows slowly (~1-2 threads/second) — can't keep up with incoming requests.

```
ThreadPool (16 threads on 8-core):
Thread 1-16: All BLOCKED on .Result calls
→ New requests queued → queue grows → timeout
```

How to confirm:

```
dotnet-counters monitor --counters System.Runtime
# Watch: ThreadPool Queue Length, ThreadPool Thread Count
```

Fix:

1. Convert all sync-over-async to proper `async/await`.
2. Use `async` database calls (`FetchAsync`, `ExecuteAsync`).
3. Never call `.Result` or `.Wait()` in request-handling code.

What to listen for: Immediately suspects ThreadPool starvation from low CPU + high latency pattern.

Q32. Explain thread safety in C#. What are the options?

Expected Answer:

```
// PROBLEM: Race condition
private int _count;
public void Increment()
{
    _count++;  // Read → Modify → Write (NOT atomic)
}
```

Solutions (from lightest to heaviest):

```
// 1. Interlocked — best for simple atomic operations
Interlocked.Increment(ref _count);

// 2. lock — mutual exclusion for code blocks
private readonly object _lock = new();
lock (_lock) { _count++; /* ... more operations */ }

// 3. SemaphoreSlim — async-compatible locking
private readonly SemaphoreSlim _semaphore = new(1, 1);
await _semaphore.WaitAsync();
try { _count++; }
finally { _semaphore.Release(); }

// 4. Concurrent collections — thread-safe data structures
ConcurrentDictionary<string, int> cache = new();
cache.AddOrUpdate("key", 1, (_, old) => old + 1);

// 5. Immutability — no shared mutable state = no race conditions
var newList = existingList.Add(item);  // ImmutableList<T>
```

Deadlock scenario:

```
// Thread 1: lock(A) → lock(B)
// Thread 2: lock(B) → lock(A)
// → Deadlock! Both waiting for each other.
// Fix: Always lock in the same order.
```

Q33. Structs vs classes — when should you use structs? What are the dangers?

Expected Answer:

Use structs when ALL true:

- Represents a single value (like `Point`, `DateTime`)
- Small (< 16 bytes)
- Immutable
- Short-lived
- Won't be boxed frequently

Danger: Mutable struct trap:

```
public struct MutablePoint
{
    public int X;
    public void MoveRight() => X++;
}

var points = new List<MutablePoint> { new() { X = 1 } };
// points[0].MoveRight();  // WON'T COMPILE — indexer returns a copy!

foreach (var p in points)
{
    p.MoveRight();  // Modifies LOCAL COPY, not the list element!
}
```

Best practice: Always use `readonly struct`:

```
public readonly struct Point
{
    public double X { get; }
    public double Y { get; }
    public Point(double x, double y) => (X, Y) = (x, y);
    public Point MoveRight() => new(X + 1, Y);  // Return new instance
}
```

Q34. What are `Span<T>` and `Memory<T>`? When would you use them?

Expected Answer:

`Span<T>` is a **stack-only view** over contiguous memory — arrays, strings, native memory — without allocation.

```
// Parse a string without creating substrings (zero allocation)
ReadOnlySpan<char> text = "Hello World".AsSpan();
ReadOnlySpan<char> hello = text[..5];   // No new string created!
ReadOnlySpan<char> world = text[6..];   // Just a pointer + length

// Slice an array without copying
int[] numbers = { 1, 2, 3, 4, 5 };
Span<int> slice = numbers.AsSpan(1, 3);  // [2, 3, 4] — no copy
slice[0] = 20;  // Modifies original array!
```

`Span<T>` **vs** `Memory<T>`**:**

- `Span<T>` — stack-only (`ref struct`), can't be used in async methods or stored in fields.
- `Memory<T>` — can be stored in fields, used in async code. Slightly more overhead.

When to use: High-performance parsing, string processing, buffer handling — anywhere you want to avoid heap allocations.

Q35. What are records? Pattern matching? What's new in modern C# (9-12)?

Expected Answer:

Records (C# 9):

```
public record Person(string Name, int Age);  // Immutable by default

var p1 = new Person("Alice", 30);
var p2 = new Person("Alice", 30);
p1 == p2;  // True! Value equality (unlike class)

var p3 = p1 with { Age = 31 };  // Non-destructive mutation
```

Pattern matching (evolved through C# 7-12):

```
// Switch expression (C# 8)
string Classify(int temp) => temp switch
{
    < 0 => "Freezing",
    >= 0 and < 20 => "Cold",
    >= 20 and < 30 => "Nice",
    >= 30 => "Hot",
};

// Property pattern
if (person is { Age: > 18, Name: not null })
    Console.WriteLine("Valid adult");

// List pattern (C# 11)
```

```
int[] nums = { 1, 2, 3 };
if (nums is [1, _, 3])
    Console.WriteLine("Starts with 1, ends with 3");
```

Other modern features:

- `init` properties (C# 9) — set only during initialization
- `required` members (C# 11) — must be set in object initializer
- Primary constructors for classes (C# 12)
- Raw string literals `"""..."""` (C# 11)
- Global using directives (C# 10)

Q36. Scenario: You're reviewing code and find this. What are the issues?

```
public class CacheService
{
    private static Dictionary<string, object> _cache = new();

    public void Set(string key, object value) => _cache[key] = value;
    public object Get(string key) => _cache.ContainsKey(key) ? _cache[key] : null;
    public void Clear() => _cache.Clear();
}
```

Expected Answer (multiple issues):

1. **Thread safety:** `Dictionary<string, object>` is not thread-safe. Concurrent reads/writes will corrupt it. → Use `ConcurrentDictionary<string, object>`.
2. **Race condition in Get:** `ContainsKey` + indexer is two operations — key could be removed between them. → Use `TryGetValue`.
3. **No expiry:** Static dictionary grows forever → memory leak. → Add TTL or use `MemoryCache`.
4. **Boxing:** `object` values box value types. → Consider `ConcurrentDictionary<string, T>` with generics.
5. **No size limit:** Unbounded cache can exhaust memory. → Add max size with eviction policy.

```
// Better:
public class CacheService
{
    private readonly IMemoryCache _cache;

    public CacheService(IMemoryCache cache) => _cache = cache;

    public void Set<T>(string key, T value, TimeSpan ttl)
        => _cache.Set(key, value, ttl);
```

```
    public T? Get<T>(string key)
        => _cache.TryGetValue(key, out T? value) ? value : default;
}
```

Q37. What is reflection and when would you use it (and when would you avoid it)?

Expected Answer:

Reflection lets you inspect and invoke types, methods, and properties **at runtime**.

```
// Get type info at runtime
Type type = typeof(Person);
PropertyInfo[] props = type.GetProperties();

foreach (var prop in props)
    Console.WriteLine($"{prop.Name}: {prop.GetValue(person)}");

// Create instance dynamically
object instance = Activator.CreateInstance(type);

// Invoke method by name
MethodInfo method = type.GetMethod("Calculate");
method.Invoke(instance, new object[] { 42 });
```

When to use: Serialization frameworks, DI containers, plugin systems, custom attribute processing.

When to avoid: Performance-critical paths — reflection is ~100x slower than direct calls. Use source generators or compiled expressions instead.

↑ Back to Index

GATE 4 — Architecture & Leadership [Lead/Architect: 15+ years]

> **Question style:** All scenario/design-based. Open-ended problems requiring trade-off analysis, production experience, and strategic thinking. **Pass criteria:** Candidate drives the discussion, asks clarifying questions, evaluates trade-offs.

Q38. Scenario: You're the tech lead for a .NET Framework 4.8 application with 200+ developers. The CTO wants to migrate to .NET 8. How do you approach this?

Expected Answer (should cover):

Assessment phase:

- Inventory all dependencies (NuGet packages, COM interop, WCF clients, System.Web usage).
- Identify blockers: APIs that don't exist in .NET 8, third-party libraries without .NET 8 support.
- Use `.NET Upgrade Assistant` and `ApiPort` tool for compatibility analysis.

Strategy options:

1. **Big bang** — Migrate everything at once. High risk, only for small apps.
2. **Strangler fig** — New features on .NET 8, gradually migrate old features. Run both side-by-side behind a reverse proxy.
3. **Incremental library-first** — Migrate shared libraries to `netstandard2.0` first, then migrate apps one by one.

Execution plan:

1. Move shared libraries to .NET Standard 2.0 (compatible with both Framework and Core).
2. Set up CI/CD for .NET 8 builds alongside Framework builds.
3. Migrate one low-risk service first as a pilot.
4. Establish migration patterns and document learnings.
5. Migrate remaining services in priority order.
6. Decommission .NET Framework builds.

What to listen for: Thinks about people (200 devs need training), process (incremental delivery), and technology (dependency compatibility). Asks clarifying questions about the app's architecture. **Red flag:** Proposes big-bang migration or underestimates complexity.

Q39. How does JIT tiered compilation work in modern .NET? Why does it matter?

Expected Answer:

```
Method first called → Tier 0 (quick compile, minimal optimization)
                          ↓
After ~30 calls     → Tier 1 (full optimization, inlining, loop unrolling)
```

Tier 0: Quick JIT — gets the app running fast (startup time). **Tier 1:** Optimized JIT — hot methods get re-compiled with full optimizations.

Why it matters:

- **Better startup time** — Don't pay optimization cost for cold methods.
- **Better steady-state performance** — Hot paths get full optimization.
- **Dynamic PGO (Profile-Guided Optimization)** in .NET 7+ — JIT uses runtime profiling data for even better optimizations (devirtualization, hot/cold path splitting).

Configuration:

```
<!-- Disable tiered compilation (rare — benchmarking scenarios) -->
<TieredCompilation>false</TieredCompilation>
```

```
<!-- Enable Dynamic PGO -->
<TieredPGO>true</TieredPGO>
```

Q40. Scenario: You're designing the performance profiling strategy for a production .NET API that handles financial transactions. What's your approach?

Expected Answer:

Instrumentation layers:

1. **Application metrics** (always on):
   - `dotnet-counters` / Prometheus metrics: request rate, error rate, GC collections, ThreadPool queue length.
   - Custom metrics: transaction processing time, cache hit ratio.
2. **Distributed tracing** (always on, sampled):
   - OpenTelemetry for request tracing across services.
   - Correlation IDs for log aggregation.
3. **Profiling (on-demand, production-safe):**
   - `dotnet-trace` for CPU profiling without restart.
   - `dotnet-dump` for memory analysis.
   - `dotnet-gcdump` for GC heap snapshots.
4. **Load testing (pre-production):**
   - `BenchmarkDotNet` for micro-benchmarks (method-level).
   - k6 or NBomber for load testing (system-level).

Alerting thresholds (for financial APIs):

- p99 latency > 500ms → warning
- p99 latency > 2s → critical
- Error rate > 0.1% → critical
- ThreadPool queue length growing → investigation

What to listen for: Layered approach (not just "add logging"), production-safe tools, understanding of the difference between profiling (overhead) and monitoring (lightweight).

Q41. GC tuning — when and how would you tune the garbage collector?

Expected Answer:

First rule: Don't tune GC until you've profiled and confirmed GC is the bottleneck.

GC Modes:

Mode	Behavior	Best for
Workstation	Concurrent, low pause	Desktop apps, low-latency
Server	One heap per CPU, parallel	Web servers, throughput

Tuning options:

```
<PropertyGroup>
    <ServerGarbageCollection>true</ServerGarbageCollection>
    <ConcurrentGarbageCollection>true</ConcurrentGarbageCollection>
    <GarbageCollectionRetainVM>true</GarbageCollectionRetainVM>
</PropertyGroup>
```

Advanced (.NET 7+):

- `GCConserveMemory` — Trade throughput for lower memory usage.
- `GCHighMemPercent` — Set threshold for aggressive collection.
- Region-based GC — Better Gen2/LOH management.

When to tune:

- GC pause times causing latency spikes (→ lower LOH threshold or enable LOH compaction).
- Memory pressure but low collection rate (→ lower `GCHighMemPercent`).
- Container environments with memory limits (→ set `GCHeapHardLimit`).

Scenario: "*Our API runs in a Docker container with 512MB limit. We're getting OOMKilled but the app only uses 300MB of managed heap.*"

Answer: Unmanaged memory (native allocations, memory-mapped files) + GC overhead. Set `GCHeapHardLimit` to ~70% of container limit. Use `dotnet-gcdump` to analyze what's on the heap.

Q42. How would you design a plugin architecture in .NET?

Expected Answer:

Using `AssemblyLoadContext` (modern approach):

```
// 1. Define plugin interface in shared assembly
public interface IPlugin
{
    string Name { get; }
    void Execute(PluginContext context);
}

// 2. Custom AssemblyLoadContext for isolation
public class PluginLoadContext : AssemblyLoadContext
{
    private readonly AssemblyDependencyResolver _resolver;
```

```
    public PluginLoadContext(string pluginPath) : base(isCollectible: true)
    {
        _resolver = new AssemblyDependencyResolver(pluginPath);
    }

    protected override Assembly? Load(AssemblyName name)
    {
        string? path = _resolver.ResolveAssemblyToPath(name);
        return path != null ? LoadFromAssemblyPath(path) : null;
    }
}

// 3. Load and execute plugins
var context = new PluginLoadContext(pluginDllPath);
var assembly = context.LoadFromAssemblyPath(pluginDllPath);
var pluginTypes = assembly.GetTypes().Where(t =>
typeof(IPlugin).IsAssignableFrom(t));

foreach (var type in pluginTypes)
{
    var plugin = (IPlugin)Activator.CreateInstance(type)!;
    plugin.Execute(appContext);
}

// 4. Unload when done (collectible context)
context.Unload();
```

Key design decisions:

- **Isolation:** Each plugin in its own `AssemblyLoadContext` — prevents version conflicts.
- **Collectible:** `isCollectible: true` allows unloading plugins.
- **Shared interface:** Plugin interface in a shared assembly both host and plugin reference.
- **Security:** Validate plugin assemblies, sandbox with limited permissions.

Q43. Scenario: "We should rewrite the system in [language X]." How do you evaluate this as an architect?

Expected Answer (framework for evaluation):

Never answer this emotionally. Use a decision framework:

1. **What problem are we solving?**
    - Performance? (Maybe tune first — profiling often finds 10x wins without rewriting.)
    - Hiring? (Valid — but retraining 50 developers has cost too.)
    - Technical debt? (Rewriting creates new debt. Incremental improvement is usually better.)
2. **What's the total cost?**
    - Rewrite: 12-24 months of parallel development.

    - Feature freeze risk: Competitors ship while you rewrite.
    - Knowledge loss: Undocumented business rules in the old code.
    - Migration: Data migration, client migration, rollback plan.

3. **Alternatives to full rewrite:**
    - Strangler fig: Replace piece by piece.
    - Optimize: Profile and fix the actual bottlenecks.
    - Modernize incrementally: Migrate to .NET 8, adopt new patterns gradually.

4. **When a rewrite IS justified:**
    - Platform is truly end-of-life (no security updates).
    - Fundamental architecture mismatch (monolith in a microservices world, AND strangler fig won't work).
    - Team has zero expertise in the current stack AND can't hire for it.

What to listen for: Asks "why?" before jumping to conclusions. Evaluates business impact, not just technology preferences. Has done this before and can share war stories. **Red flag:** Strong opinion without framework. "We should use Rust because it's faster."

APPENDIX — Quick Reference Card

C# Keywords Cheat Sheet

```
Access:     public | private | protected | internal | protected internal | private
protected
Types:      class | struct | interface | enum | record | delegate
Modifiers:  static | abstract | virtual | override | sealed | new | readonly |
const
Async:      async | await
Null:       null | ?? | ?. | ??= | ! (null-forgiving)
Pattern:    is | switch expression | when | and | or | not
Memory:     ref | out | in | params | stackalloc | Span<T>
```

One-Liner Answers

Question	Quick Answer
`const` vs `readonly`	`const` = compile-time, inlined; `readonly` = runtime, set in constructor
`sealed`	Cannot be inherited — enables devirtualization (JIT optimization)
`record` vs `class`	Record: value equality, immutable by default, `with` expressions
`is` vs `as`	`is` = type check (bool); `as` = safe cast (null on failure, no exception)
`ref` vs `out`	`ref` = must initialize before passing; `out` = must assign inside the method

Question	Quick Answer
`throw` vs `throw ex`	`throw` preserves stack trace; `throw ex` resets it
`String.Empty` vs `""`	Identical — both reference the same interned string
`IEnumerable` vs `ICollection`	`IEnumerable` = iterate only; `ICollection` adds Count, Add, Remove
`Task` vs `Thread`	`Task` = abstraction over ThreadPool (lightweight); `Thread` = OS thread (expensive)
`ConfigureAwait(false)`	Don't capture SynchronizationContext — use in library code

> **Gate Assessment Summary:**
>
> | Gate | Passed | Level |
> |---|---|---|
> | Gate 1 | Yes | At least Junior |
> | Gate 2 | Yes | At least Mid |
> | Gate 3 | Yes | At least Senior |
> | Gate 4 | Yes | Lead/Architect ready |
>
> **Tip for interviewer:** Don't skip gates even for 20-year candidates. Start at Gate 1 — you'll learn a lot from HOW they answer basic questions (depth of explanation, real-world examples, nuance).

⬆ Back to Index